R workshop part 6 - Complete project demonstration

Jan Gačnik

Prepared: 2026-10-26

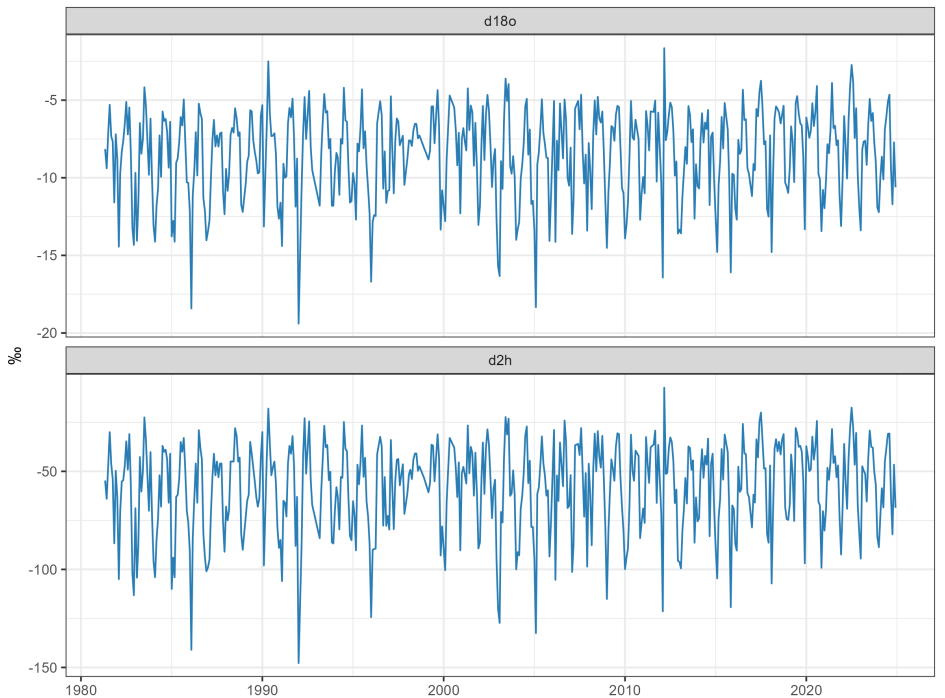
One project, five parts

In this final demonstration, we will connect the main skills from the workshop in one reproducible analysis.

1. Import and inspect data
2. Select, rename, and filter
3. Group and summarize
4. Reshape and visualize
5. Model and save results

Follow the slides, but run each section in `scripts/project_demonstration.R` in RStudio.

The demonstration data



We will analyze `precipitation_ljubljana.xlsx`:

- monthly precipitation measurements from Ljubljana;
- 44 years of observations, from 1981 to 2024;
- precipitation amount, temperature, humidity, and isotope values;
- several Excel sheets containing data and supporting information.

Start inside the project

Extract `example_project.zip` and open `example_project.Rproj`.
Then open the demonstration script.

```
example_project/
├── data/
│   └── precipitation_ljubljana.xlsx
├── scripts/
│   └── project_demonstration.R
└── outputs/
```

The project makes relative paths such as `data/...` work on every computer.

1. Import and inspect

Run sections 1 and 2. This recalls **objects, functions, packages, and data frames**.

```
library(tidyverse)
library(readxl)

ljubljana_raw <- read_xlsx(
  "data/precipitation_ljubljana.xlsx",
  sheet = "Data Ljubljana"
)

glimpse(ljubljana_raw)
head(ljubljana_raw)
```

2. Prepare the data

Run section 3. Keep only the variables needed for the analysis and give them clear names.

```
ljubljana_clean <- ljubljana_raw %>%
  select(Year, Month, P, T, RH, d18O, d2H) %>%
  rename(
    year = Year,
    month = Month,
    precipitation_mm = P,
    d18o = d18O,
    d2h = d2H
  ) %>%
  mutate(date = lubridate::make_date(year, month, 1))

isotope_data <- ljubljana_clean %>%
  filter(!is.na(d18o), !is.na(d2h))
```

3. Group and summarize

Run section 4. Change the level of observation from **one month** to **one year**.

```r
annual_summary <- ljubljana_clean %>%
  group_by(year) %>%
  summarise(
    months_with_isotopes = sum(!is.na(d18o) & !is.na(d2h)),
    mean_d18o = mean(d18o, na.rm = TRUE),
    mean_d2h = mean(d2h, na.rm = TRUE),
    annual_precipitation_mm = sum(precipitation_mm, na.rm = TRUE),
    .groups = "drop"
  )

head(annual_summary)
```

4. Reshape for plotting

Run section 5. Put both isotope measurements into the same pair of columns.

```
isotopes_long <- isotope_data %>%
  select(date, d18o, d2h) %>%
  pivot_longer(
    cols = c(d18o, d2h),
    names_to = "isotope",
    values_to = "isotope_value"
  )
```

Each row now contains one date, one isotope name, and one measured value: **tidy data**.

5. Visualize

Run section 6. The long data make it easy to use the same plot design for both isotopes.

```
isotope_time_plot <- isotopes_long %>%
  ggplot(aes(x = date, y = isotope_value)) +
  geom_line(color = "#2C7FB8", linewidth = 0.5) +
  facet_wrap(~ isotope, ncol = 1, scales = "free_y") +
  labs(
    title = "Precipitation isotopes through time",
    x = NULL,
    y = "Isotope value"
  ) +
  theme_minimal()

print(isotope_time_plot)
```

6. Model and save

Run section 7 to describe the relationship between the two isotopes.

```
isotope_model <- lm(
  d2h ~ d18o,
  data = isotope_data
)

summary(isotope_model)
```

Run section 8 to save results in the project.

```
write_csv(
  annual_summary,
  "outputs/annual_isotope_summary.csv"
)

ggsave(
  "outputs/isotopes_through_time.png",
  isotope_time_plot
)
```

Your turn

Run the full script from top to bottom. Then make **one small change**:

- analyze only observations from 2000 onwards;
- change a color or label in one plot; or
- add one more statistic to `annual_summary`.

Re-run the affected sections and check the `outputs` folder.

A complete, reproducible workflow

The demonstration joined the workshop parts into one script:

- **Part 1:** objects, data frames, functions, and missing values
- **Part 2:** plots and a simple linear model
- **Part 3:** selecting, filtering, grouping, and summarizing
- **Part 4:** tidy data and reshaping
- **Part 5:** projects, relative paths, importing, and exporting

Because the inputs, code, and outputs stay together, the analysis can be repeated and shared.